

State Management

1. State – ref()

First, we need to store the student's name.

```
<script setup>  
import { ref } from 'vue'  
const studentName = ref('Prabha')  
</script>
```

```
<template>  
  <h2>Student: {{ studentName }}</h2>  
</template>
```

Here, studentName is the **state**.

2. Reactive State

Now let the student change the name.

```
<script setup>  
import { ref } from 'vue'  
const studentName = ref('Prabha')  
</script>  
<template>  
  <input v-model="studentName">  
  <h2>Student: {{ studentName }}</h2>  
</template>
```

When the user changes the input, the displayed name automatically changes.

Concept: Vue's reactive state keeps the **data and UI synchronized**.

3. Local State

Suppose we add a course selection.

```
<script setup>
import { ref } from 'vue'
const studentName = ref('Prabha')
const course = ref('Vue.js')
</script>
<template>
  <input v-model="studentName">
  <select v-model="course">
    <option>Vue.js</option>
    <option>React</option>
    <option>Angular</option>
  </select>
  <h2>{{ studentName }}</h2>
  <p>Course: {{ course }}</p>
</template>
```

Both studentName and course are **local state** because they belong to this component.

4. State Update

Now add a **Register** button.

```
<script setup>
import { ref } from 'vue'
const studentName = ref('Prabha')
const course = ref('Vue.js')
const registered = ref(false)
function register() {
  registered.value = true
}
</script>
<template>
  <input v-model="studentName">
  <select v-model="course">
    <option>Vue.js</option>
    <option>React</option>
    <option>Angular</option>
  </select>
```

```
<button @click="register"> Register </button>
<p v-if="registered">
  {{ studentName }} registered for {{ course }}
</p>
```

```
</template>
```

Here:

registered.value = true
is a **state update**.

5. Computed State

Now suppose we want to display a message based on the selected course.

```
<script setup>
import { ref, computed } from 'vue'
const studentName = ref('Prabha')
const course = ref('Vue.js')
const courseMessage = computed(() => {
  return studentName.value + ' selected ' + course.value
})
</script>
<template>
  <input v-model="studentName">

  <select v-model="course">
    <option>Vue.js</option>
    <option>React</option>
    <option>Angular</option>
  </select>
  <p>{{ courseMessage }}</p>
</template>
```

courseMessage is **not independent data**. It is calculated from existing state.

6. Shared State Between Components

Now our application becomes slightly larger:

App.vue

- Student.vue
- Course.vue

store.js

```
import { ref } from 'vue'
```

```
export const selectedCourse = ref('Vue.js')
```

Suppose both components need to know the **selected course**.

Create a simple shared state:

Student.vue:

```
<script setup>
```

```
import { selectedCourse } from './store.js'
```

```
</script>
```

```
<template>
```

```
  <h2>Selected Course:
```

```
{{ selectedCourse }}</h2>
```

```
</template>
```

Course.vue:

```
<script setup>
```

```
import { selectedCourse } from './store.js'
```

```
</script>
```

```
<template>
```

```
  <button @click="selectedCourse = 'React'">
```

```
    Select React
```

```
  </button>
```

```
</template>
```

Now both components use the **same state**.

When the application becomes larger, instead of manually creating our own store, Vue applications commonly use **Pinia**.

The same application could have:

```
import { defineStore } from 'pinia'
```

```
export const useStudentStore = defineStore('student', {  
  state: () => ({  
    studentName: 'Prabha',  
    course: 'Vue.js',  
    registered: false  
  }),  
  actions: {  
    register() {  
      this.registered = true  
    }  
  }  
})
```

A component can use it:

```
<script setup>
import { useStudentStore } from './stores/student' const
student = useStudentStore()
</script>
```

```
<template>
<h2>{{ student.studentName }}</h2>
<p>{{ student.course }}</p>
<button @click="student.register"> Register </button>
</template>
```

Question: Develop a Vue.js Online Food Ordering application using Pinia for state management.

The application should allow the user to:

1. Display a list of food items:
 1. 🍕 Pizza – ₹250
 2. 🍔 Burger – ₹150
 3. 🍝 Pasta – ₹200
 4. ☐ Coke – ₹80
2. Allow the user to click **Add to Cart**.
3. Display the selected items in the cart.
4. Allow the user to **increase and decrease the quantity**.
5. Display:
 1. Number of items in the cart.
 2. Total price.
6. Use a **Pinia store** to maintain:
 1. cart
 2. totalItems
 3. totalPrice

7. Define Pinia actions:

1. addToCart()
2. increaseQuantity()
3. decreaseQuantity()
4. removeFromCart()
5. clearCart()

8. Use Pinia getters to calculate the **total number of items** and **total bill amount**.

Expected Output

☐ QUICK BITE

Available Food

🍕 Pizza ₹250 [Add]
🍔 Burger ₹150 [Add]
🍝 Pasta ₹200 [Add]
☐ Coke ₹80 [Add]

🛒 Cart (3 items)

🍕 Pizza [-] 2 [+] ₹500
☐ Coke [-] 1 [+] ₹80

Total: ₹580

[Clear Cart]